UNIVERSITY OF RUHUNA

Faculty of Engineering

End-Semester 3 Examination in Engineering: December 2023

Module Number: EE3302 **Module Name: Data Structures and Algorithms**

[Three Hours]

[Answer **all questions**, each question carries 10 marks]

Q1. a) An array is a linear data structure that stores elements in adjacent memory locations.

i) State two (2) types of memory that are used to store variables. Explain a key difference between them.

[1.0 mark]

ii) Write a C++ code to reverse a given integer array.

[1.5 marks]

iii) Compute the time complexity of the code written in Q1.a).ii). (use big O notation)

[1.0 mark]

b) A singly linked list stores elements in nodes, with each node pointing to the next in the sequence.

i) Describe an application where a linked list would be more suitable than an array.

[1.0 mark]

ii) Explain how to implement a sorted Singly Linked List. The data should be inserted in ascending order from the head of the list.

[1.0 mark]

iii) Write a C++ program to implement the insert operation explained in Q1.b).ii). Note that data should be inserted into the Singly Linked List while maintaining the ascending order.

[2.0 marks]

c) A Doubly Linked List has links in both forward and backward directions.

i) Write a C++ program to delete the last node in a Doubly Linked List.

[1.5 marks]

ii) Compute the time complexity of the delete operation in part Q1.c).i). (use big O notation).

[1.0 mark]

Q2. a) A Tree is a non-linear data structure, which can store values in scattered memory locations.

i) Insert the following numbers in the given order to an empty Binary Search Tree (BST).

53, 79, 91, 110, 141, 85, 2, 41, 88

[1.5 marks]

ii) Insert the same numbers in Q2.a).i) in the given order to an empty AVL tree. You must draw the resulting tree after each insertion.

[2.0 marks]

iii) Compare the trees in Q2.a).i) and Q2.a).ii) and discuss the use cases of each tree.

[1.5 marks]

iv) Write the order of the nodes printed in **Pre-order** traversal, when applied on the AVL tree created in part Q2.a).ii).

[1.0 mark]

b) Hash table facilitates efficient data retrieval and storage by mapping keys to corresponding values through a hash function.

i) Briefly explain the closed addressing collision handling technique used in Hash tables.

[1.0 mark]

ii) Comment on how the time complexity changes when closed addressing is used to resolve collisions.

[1.0 mark]

iii) The NIC numbers of the sitcom *Friends* are given in Table Q2.b. Insert the entries in a Hash table of size 10. The Hash function will sum the value of each digit of the NIC number and take the modules out of the size of the table to generate the index. Show the workings and the final Hash table. Use **Plus 3 rehash** as the collision resolution method.

[2.0 marks]

Table Q2.b: NIC numbers and names of the sitcom *Friends*

NIC	Name
9314078652	Rachel
9743102863	Ross
9652087314	Chandler
9205748562	Monica
9832160375	Joey
9187162043	Phoebe
9045726183	Gunther

Q3. a) Stack and Queue are two logical data structures.

i) State two (2) real world applications of Stack and Queue data structures each.

[1.0 mark]

ii) Explain how to sort a stack using another temporary stack, providing a working example. You may use a variable to store temporary values.

[2.0 marks]

iii) Write a C++ program to implement the enQueue operation of the Queue data structure. Assume *rear* variable is pointing to the last element with a value.

[1.5 marks]

iv) What is the time complexity of the enQueue operation implemented in Q3.a).iii)? (use big O notation)

[0.5 marks]

b) Algorithm analysis is the process of evaluating the performance of an algorithm in terms of the amount of time and resources it requires to execute. Find the time complexity of the following code snippets with respect to input size N.

i)

```
void function(int N) {
    int i = 1;
    while (i <= N) {
        i = i * 2;
    }
}
```

[1.0 mark]

ii)

```
void function(int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << i << "␣" << j << endl;
        }
    }
}
```

[1.0 mark]

iii)

```
int function(int N) {
    if (N == 1) {
        return N;
    } else {
        return N * function(N-1);
    }
}
```

[1.0 mark]

iv)

```
void function(int N) {
    int i = 1;
    while (i < N) {
        i = i + 2;
    }
}
```

[1.0 mark]

v)

```
void function(int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < N; k++) {
                cout << i << j << k << endl;
            }
        }
    }
}
```

[1.0 mark]

Q4. a) i) Explain the Bubble Sort algorithm given below by using a diagram for a given array of elements.

```
void bubbleSort(int arr[], int N) {
    for (int i = 0; i < N-1; i++) {
        for (int j = 0; j < N-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

[2.0 marks]

ii) The Bubble Sort algorithm can be optimized by stopping the algorithm if the inner loop does not cause any swap in the last iteration. Modify the Bubble Sort code snippet in Q4.a).i) to include this optimization.

[2.0 marks]

iii) What is the best-case running time of the optimized algorithm in Q4.a).ii)?

[1.0 mark]

b) The Merge Sort algorithm is a divide-and-conquer sorting algorithm.

i) Explain in detail how the Merge Sort algorithm works. Include a step-by-step walkthrough of the algorithm with a sample input array.

[2.0 marks]

ii) Analyze the time complexity of the Merge Sort algorithm. Discuss the best, average, and worst-case scenarios and explain why these cases yield different time complexities.

[1.5 marks]

iii) The Merge Sort algorithm requires extra space for the Left and Right arrays in the merge function. Discuss the space complexity of the Merge Sort algorithm and how it compares to Quick Sort algorithms.

[1.5 marks]

Q5. a) Consider the following adjacency list representation of a directed graph.

```
1: 2, 3, 4
2: 1, 3, 5
3: 2, 4, 6
4: 1, 3, 5, 7
5: 2, 4, 6, 8
6: 3, 5, 7, 1
7: 4, 6, 8, 2
8: 5, 7, 1, 3
```

This representation indicates that there is a directed edge from vertex 1 to vertices 2, 3, and 4, a directed edge from vertex 2 to vertices 1, 3, and 5, and so on. (The first number is the starting node of each line, the rest are the destination nodes of the edges)

i) Based on this adjacency list, draw the corresponding directed graph. Label each vertex with its corresponding number and draw an arrow from vertex *i* to vertex *j* if there is an edge.

[2.0 marks]

ii) Draw the corresponding adjacency matrix representation of this graph. Label each row and column with its corresponding vertex number and fill in the cell with a 1 if there is a directed edge and with a 0 otherwise.

[2.0 marks]

b) Apply the Depth-First Search (DFS) algorithm to the graph in Q5.a), starting from vertex 1. List the vertices in the order they are visited.

[2.0 marks]

c) Apply the Breath-First-Search (BFS) algorithm to the graph in Q5.a), starting from vertex 1. List the vertices in the order they are visited.

[2.0 marks]

d) What are the time complexity of the BFS and DFS algorithms?

[2.0 marks]